

ASSESSMENT TOOL CO1 AT1

SUTDENT NAME: P. VISHNU VARDHAN REDDY

REG. NO: 192425212

COURSE CODE: MLA0302

COURSE NAME: REINFORCEMENT LEARNING

Experiment 1: Installation of RL Development Environment

Aim :

To install and configure the Reinforcement Learning (RL) development environment using Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the installation by executing a simple Reinforcement Learning program.

Procedure :

1. Download and install Anaconda Navigator on your computer.
2. Open Anaconda Prompt and create a new Python environment.
3. Install the required libraries using the pip command:
 - a. `pip install gymnasium tensorflow keras numpy matplotlib`
4. Open Spyder, Jupyter Notebook, or VS Code from the created environment.
5. Write a simple Reinforcement Learning program to create a Gymnasium environment and run a few steps.
6. Execute the program and verify that the environment runs successfully without errors.

Output:

```
*** TensorFlow Version : 2.20.0
    Keras Version      : 3.13.2
    Numpy Version      : 2.0.2
    Gymnasium Version  : 1.3.0

Initial Observation:
[0.03663001 0.00103856 0.04008184 0.03157765]

Step 1
Action: 1
Reward: 1.0
Observation: [ 0.03665078  0.19556348  0.04071339 -0.24819447]

Step 2
Action: 1
Reward: 1.0
Observation: [ 0.04056205  0.39008105  0.0357495  -0.5277626 ]

Step 3
Action: 1
Reward: 1.0
Observation: [ 0.04836367  0.5846823  0.02510425 -0.6089698 ]

CartPole-v1 Environment
```



Experiment Completed Successfully!

Result:

The Reinforcement Learning development environment was successfully installed and configured. The required libraries were verified, and a simple RL program executed successfully.

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

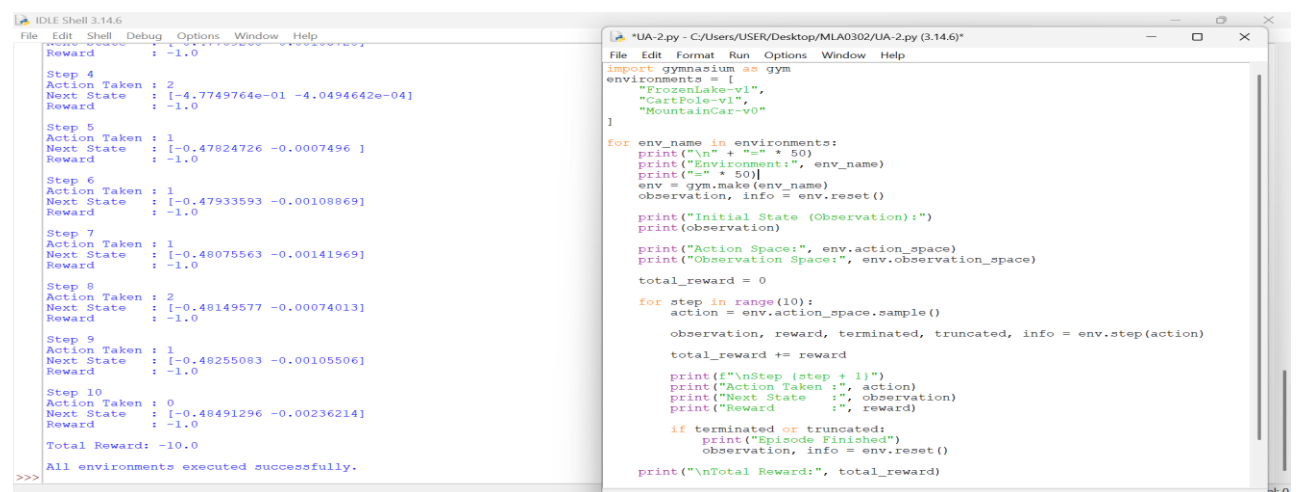
Aim:

To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Procedure :

1. Import the Gymnasium library.
2. Create the FrozenLake, CartPole, and MountainCar environments.
3. Initialize each environment using the reset() function.
4. Execute random actions using the action space.
5. Observe the state transitions, rewards, and episode termination.
6. Close the environment after execution.

Code :



The image shows two side-by-side windows from an IDE. The left window, titled 'IDLE Shell 3.14.6', displays the output of a Python script. It shows a series of steps (Step 4 to Step 10) where actions are taken, next states are observed, and rewards are calculated. The final output is 'Total Reward: -10.0' and 'All environments executed successfully.' The right window, titled 'UA-2.py - C:/Users/USER/Desktop/MLA0302/UA-2.py (3.14.6)', shows the Python code that generates this output. The code imports the 'gym' library, defines a list of environments ('FrozenLake-v1', 'CartPole-v1', 'MountainCar-v0'), and then iterates over them. For each environment, it prints the environment name, resets it, prints the initial state and action space, and then performs a series of random actions (up to 10 steps) before printing the final state, action, and reward. The total reward across all environments is printed at the end.

```
File Edit Shell Debug Options Window Help
>>>
Step 4
Action Taken : 2
Next State : [-4.7749764e-01 -4.0494642e-04]
Reward : -1.0

Step 5
Action Taken : 1
Next State : [-0.47824726 -0.0007496 ]
Reward : -1.0

Step 6
Action Taken : 1
Next State : [-0.47933593 -0.00108869]
Reward : -1.0

Step 7
Action Taken : 1
Next State : [-0.48075563 -0.00141969]
Reward : -1.0

Step 8
Action Taken : 2
Next State : [-0.48149577 -0.00074013]
Reward : -1.0

Step 9
Action Taken : 1
Next State : [-0.48255083 -0.00105506]
Reward : -1.0

Step 10
Action Taken : 0
Next State : [-0.48491296 -0.00236214]
Reward : -1.0

Total Reward: -10.0
All environments executed successfully.
>>>
```

```
File Edit Format Run Options Window Help
"UA-2.py - C:/Users/USER/Desktop/MLA0302/UA-2.py (3.14.6)"
import gymnasium as gym
environments = [
    "FrozenLake-v1",
    "CartPole-v1",
    "MountainCar-v0"
]

for env_name in environments:
    print("\n" + "=" * 50)
    print("Environment:", env_name)
    print("=" * 50)
    env = gym.make(env_name)
    observation, info = env.reset()

    print("Initial State (Observation):")
    print(observation)

    print("Action Space:", env.action_space)
    print("Observation Space:", env.observation_space)

    total_reward = 0

    for step in range(10):
        action = env.action_space.sample()
        observation, reward, terminated, truncated, info = env.step(action)

        total_reward += reward

        print(f"\nStep (step + 1)")
        print("Action Taken :", action)
        print("Next State :", observation)
        print("Reward :", reward)

    if terminated or truncated:
        print("Episode Finished")
        observation, info = env.reset()

    print("\nTotal Reward:", total_reward)
```

Result:

The FrozenLake, CartPole, and MountainCar environments were successfully created and explored. The concepts of states, actions, rewards, and episodes were understood through interaction.

Experiment 3: Implementation of the Reinforcement Learning Framework

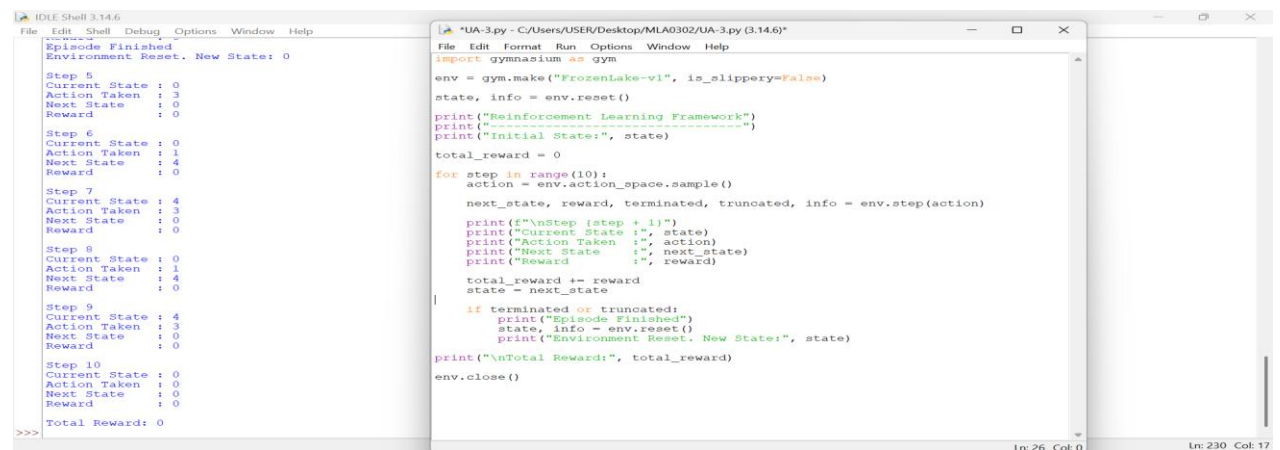
Aim:

To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Procedure :

1. Import the required Python libraries.
2. Create a Reinforcement Learning environment.
3. Initialize the environment.
4. Design an agent that selects actions.
5. Execute actions and receive rewards.
6. Update the current state until the episode ends.
7. Display the total reward obtained.

Output :



The screenshot displays two windows from the IDLE 3.14.6 environment. The left window, titled 'IDLE Shell 3.14.6', shows the output of the script, which includes the state, action, and reward for each step of an episode, followed by the total reward. The right window, titled '*UA-3.py - C:/Users/USER/Desktop/MLA0302/UA-3.py (3.14.6)*', shows the source code of the script.

```
Episode Finished
Environment Reset. New State: 0
Step 5
Current State : 0
Action Taken : 3
Next State : 0
Reward : 0
Step 6
Current State : 0
Action Taken : 1
Next State : 4
Reward : 0
Step 7
Current State : 4
Action Taken : 3
Next State : 0
Reward : 0
Step 8
Current State : 0
Action Taken : 1
Next State : 4
Reward : 0
Step 9
Current State : 4
Action Taken : 3
Next State : 0
Reward : 0
Step 10
Current State : 0
Action Taken : 0
Next State : 0
Reward : 0
Total Reward: 0
>>>
```

```
*UA-3.py - C:/Users/USER/Desktop/MLA0302/UA-3.py (3.14.6)*
import gymnasium as gym

env = gym.make("FrozenLake-v1", is_slippery=False)
state, info = env.reset()

print("Reinforcement Learning Framework")
print("-----")
print("Initial State:", state)

total_reward = 0

for step in range(10):
    action = env.action_space.sample()
    next_state, reward, terminated, truncated, info = env.step(action)

    print(f"\nStep (step + 1)")
    print("Current State :", state)
    print("Action Taken :", action)
    print("Next State :", next_state)
    print("Reward :", reward)

    total_reward += reward
    state = next_state

    if terminated or truncated:
        print("Episode Finished")
        state, info = env.reset()
        print("Environment Reset. New State:", state)

print("\nTotal Reward:", total_reward)

env.close()
```

Result:

A basic Reinforcement Learning framework was successfully implemented, demonstrating the interaction between the agent and the environment using states, actions, rewards, and policies.

Experiment 4: Simulation of a Markov Decision Process (MDP)

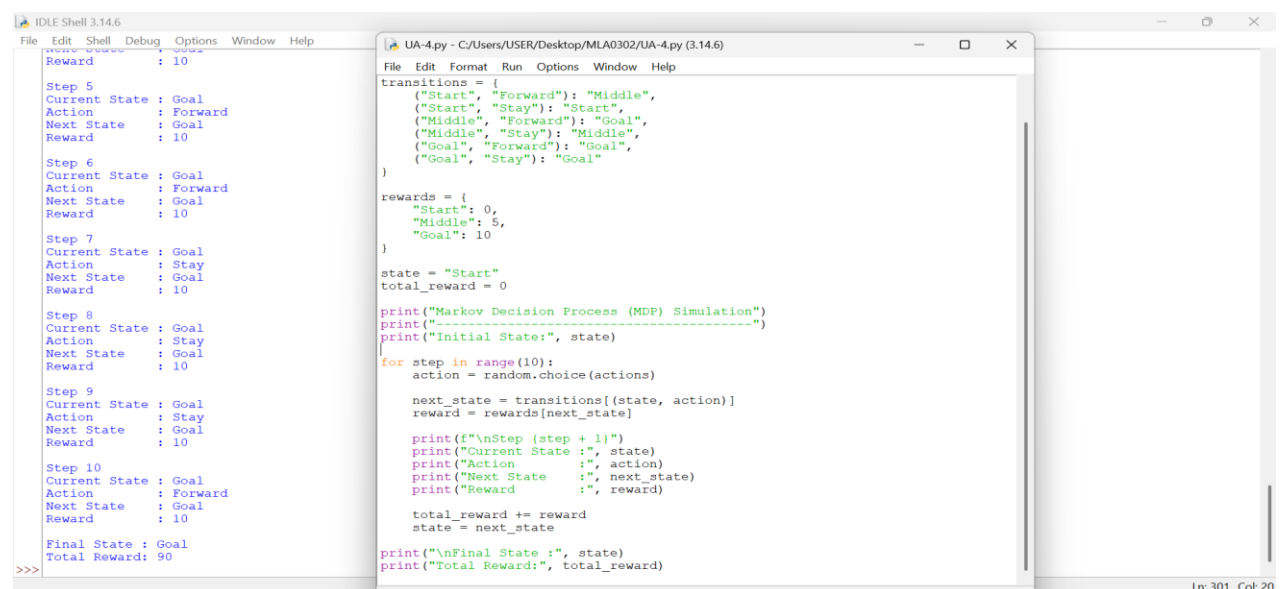
Aim:

To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Procedure :

1. Define the states and actions of the MDP.
2. Specify the transition probabilities.
3. Define the reward function.
4. Initialize the starting state.
5. Simulate state transitions by selecting actions.
6. Calculate rewards at each transition.
7. Display the final state and total reward.

output :



The image shows two windows from the IDLE 3.14.6 Python environment. The left window, titled 'IDLE Shell 3.14.6', displays the output of a Python script. The right window, titled 'UA-4.py - C:/Users/USER/Desktop/MLA0302/UA-4.py (3.14.6)', shows the source code of the script.

Output (Left Window):

```
Reward : 10
Step 5
Current State : Goal
Action : Forward
Next State : Goal
Reward : 10
Step 6
Current State : Goal
Action : Forward
Next State : Goal
Reward : 10
Step 7
Current State : Goal
Action : Stay
Next State : Goal
Reward : 10
Step 8
Current State : Goal
Action : Stay
Next State : Goal
Reward : 10
Step 9
Current State : Goal
Action : Stay
Next State : Goal
Reward : 10
Step 10
Current State : Goal
Action : Forward
Next State : Goal
Reward : 10
Final State : Goal
Total Reward: 90
>>>
```

Code (Right Window):

```
transitions = {
    ("Start", "Forward"): "Middle",
    ("Start", "Stay"): "Start",
    ("Middle", "Forward"): "Goal",
    ("Middle", "Stay"): "Middle",
    ("Goal", "Forward"): "Goal",
    ("Goal", "Stay"): "Goal"
}

rewards = {
    "Start": 0,
    "Middle": 5,
    "Goal": 10
}

state = "Start"
total_reward = 0

print("Markov Decision Process (MDP) Simulation")
print("-----")
print("Initial State:", state)

for step in range(10):
    action = random.choice(actions)

    next_state = transitions[(state, action)]
    reward = rewards[next_state]

    print(f"\nStep {step + 1}")
    print("Current State :", state)
    print("Action      :", action)
    print("Next State   :", next_state)
    print("Reward       :", reward)

    total_reward += reward
    state = next_state

print("\nFinal State :", state)
print("Total Reward:", total_reward)
```

Result:

The Markov Decision Process was successfully simulated, demonstrating state transitions, reward calculation, and policy execution.

Experiment 5: Bellman Equation Demonstration

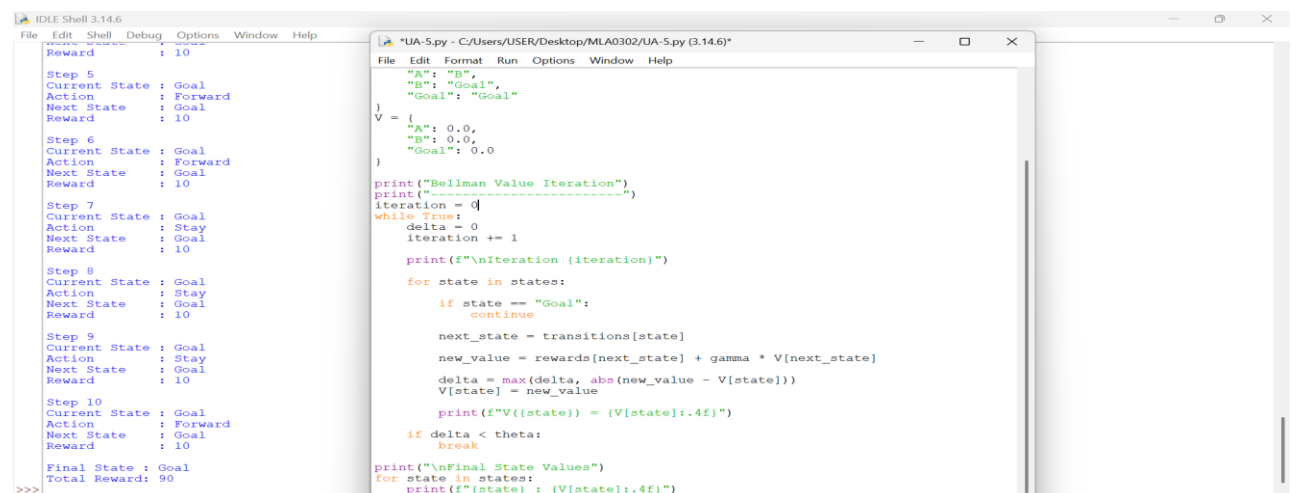
Aim:

To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

Procedure :

1. Define the states and rewards.
2. Initialize the state-value function.
3. Set the discount factor and convergence threshold.
4. Apply the Bellman update equation iteratively.
5. Update the value of each state.
6. Continue until convergence is achieved.
7. Display the final state values.

Output :



```
File Edit Shell Debug Options Window Help
Reward : 10
Step 5
Current State : Goal
Action : Forward
Next State : Goal
Reward : 10
Step 6
Current State : Goal
Action : Stay
Next State : Goal
Reward : 10
Step 7
Current State : Goal
Action : Stay
Next State : Goal
Reward : 10
Step 8
Current State : Goal
Action : Stay
Next State : Goal
Reward : 10
Step 9
Current State : Goal
Action : Stay
Next State : Goal
Reward : 10
Step 10
Current State : Goal
Action : Forward
Next State : Goal
Reward : 10
Final State : Goal
Total Reward: 90
>>>
```

```
File Edit Format Run Options Window Help
*UA-5.py - C:/Users/USER/Desktop/MLA0302/UA-5.py (3.14.6)*
"A": "B",
"B": "Goal",
"Goal": "Goal"
}
V = {
    "A": 0.0,
    "B": 0.0,
    "Goal": 0.0
}
print("Bellman Value Iteration")
print("-----")
iteration = 0
while True:
    delta = 0
    iteration += 1
    print(f"\nIteration {iteration}")
    for state in states:
        if state == "Goal":
            continue
        next_state = transitions[state]
        new_value = rewards[next_state] + gamma * V[next_state]
        delta = max(delta, abs(new_value - V[state]))
        V[state] = new_value
    print(f"V({state}) = {V[state]:.4f}")
    if delta < theta:
        break
print("\nFinal State Values")
for state in states:
    print(f"{state} : {V[state]:.4f}")
```

Result:

The Bellman Equation was successfully implemented, and the state-value function converged to the optimal values through iterative updates.

Experiment 6: Exploration vs. Exploitation Strategies

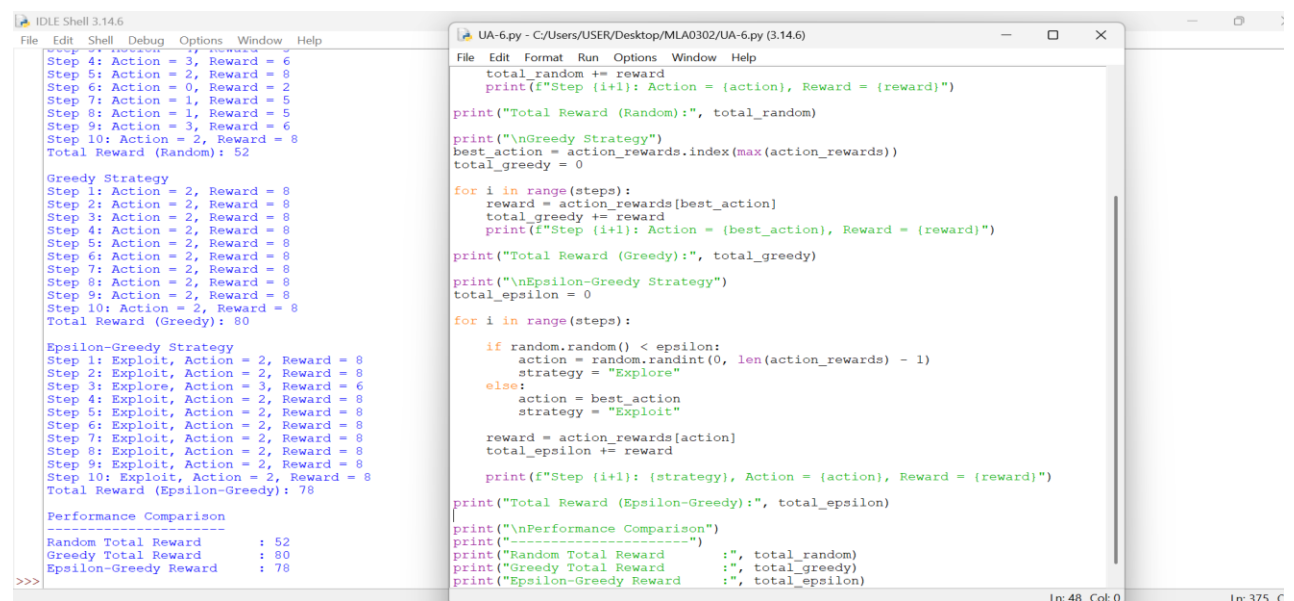
Aim:

To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Procedure :

1. Define the available actions and rewards.
2. Implement the Random action-selection strategy.
3. Implement the Greedy strategy.
4. Implement the ϵ -Greedy strategy.
5. Execute each strategy for multiple steps.
6. Calculate the total reward obtained by each strategy.
7. Compare the performance of all strategies.

Output :



```
File Edit Shell Debug Options Window Help
Step 4: Action = 3, Reward = 6
Step 5: Action = 2, Reward = 8
Step 6: Action = 0, Reward = 2
Step 7: Action = 1, Reward = 5
Step 8: Action = 1, Reward = 5
Step 9: Action = 3, Reward = 6
Step 10: Action = 2, Reward = 8
Total Reward (Random): 52

Greedy Strategy
Step 1: Action = 2, Reward = 8
Step 2: Action = 2, Reward = 8
Step 3: Action = 2, Reward = 8
Step 4: Action = 2, Reward = 8
Step 5: Action = 2, Reward = 8
Step 6: Action = 2, Reward = 8
Step 7: Action = 2, Reward = 8
Step 8: Action = 2, Reward = 8
Step 9: Action = 2, Reward = 8
Step 10: Action = 2, Reward = 8
Total Reward (Greedy): 80

Epsilon-Greedy Strategy
Step 1: Exploit, Action = 2, Reward = 8
Step 2: Exploit, Action = 2, Reward = 8
Step 3: Explore, Action = 3, Reward = 6
Step 4: Exploit, Action = 2, Reward = 8
Step 5: Exploit, Action = 2, Reward = 8
Step 6: Exploit, Action = 2, Reward = 8
Step 7: Exploit, Action = 2, Reward = 8
Step 8: Exploit, Action = 2, Reward = 8
Step 9: Exploit, Action = 2, Reward = 8
Step 10: Exploit, Action = 2, Reward = 8
Total Reward (Epsilon-Greedy): 78

Performance Comparison
Random Total Reward      : 52
Greedy Total Reward      : 80
Epsilon-Greedy Reward    : 78
>>>
```

```
File Edit Format Run Options Window Help
UA-6.py - C:/Users/USER/Desktop/MLA0302/UA-6.py (3.14.6)

total_random += reward
print(f"Step {i+1}: Action = {action}, Reward = {reward}")

print("Total Reward (Random):", total_random)

print("\nGreedy Strategy")
best_action = action_rewards.index(max(action_rewards))
total_greedy = 0

for i in range(steps):
    reward = action_rewards[best_action]
    total_greedy += reward
    print(f"Step {i+1}: Action = {best_action}, Reward = {reward}")

print("Total Reward (Greedy):", total_greedy)

print("\nEpsilon-Greedy Strategy")
total_epsilon = 0

for i in range(steps):
    if random.random() < epsilon:
        action = random.randint(0, len(action_rewards) - 1)
        strategy = "Explore"
    else:
        action = best_action
        strategy = "Exploit"

    reward = action_rewards[action]
    total_epsilon += reward

    print(f"Step {i+1}: {strategy}, Action = {action}, Reward = {reward}")

print("Total Reward (Epsilon-Greedy):", total_epsilon)

print("\nPerformance Comparison")
print("-----")
print("Random Total Reward      :", total_random)
print("Greedy Total Reward      :", total_greedy)
print("Epsilon-Greedy Reward    :", total_epsilon)
```

Ln: 48 Col: 0 Ln: 375 C

Result:

The Random, Greedy, and ϵ -Greedy strategies were successfully implemented and compared, demonstrating the trade-off between exploration and exploitation.

Experiment 7: Multi-Armed Bandit Problem

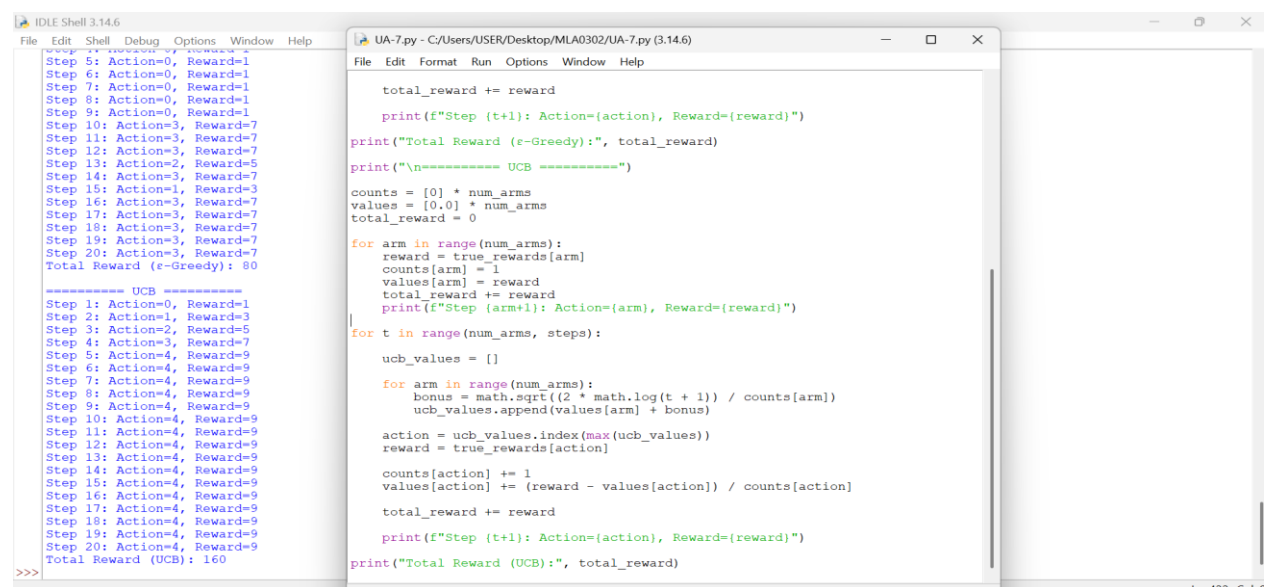
Aim:

To implement the Multi-Armed Bandit problem using ϵ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

Procedure :

1. Define multiple bandit arms with different rewards.
2. Initialize the ϵ -Greedy algorithm.
3. Execute actions and update reward estimates.
4. Implement the Upper Confidence Bound (UCB) algorithm.
5. Execute the UCB action-selection strategy.
6. Calculate cumulative rewards.
7. Compare the performance of ϵ -Greedy and UCB.

Output :



The image shows two side-by-side windows from the IDLE 3.14.6 Python IDE. The left window displays the output of a script, showing step-by-step actions and rewards for both ϵ -Greedy and UCB algorithms. The right window shows the corresponding Python code.

Left Window Output:

```
Step 5: Action=0, Reward=1
Step 6: Action=0, Reward=1
Step 7: Action=0, Reward=1
Step 8: Action=0, Reward=1
Step 9: Action=0, Reward=1
Step 10: Action=3, Reward=7
print("Total Reward ( $\epsilon$ -Greedy):", total_reward)
Step 11: Action=3, Reward=7
Step 12: Action=3, Reward=7
Step 13: Action=2, Reward=5
Step 14: Action=3, Reward=7
Step 15: Action=1, Reward=3
Step 16: Action=3, Reward=7
Step 17: Action=3, Reward=7
Step 18: Action=3, Reward=7
Step 19: Action=3, Reward=7
Step 20: Action=3, Reward=7
Total Reward ( $\epsilon$ -Greedy): 80

===== UCB =====
Step 1: Action=0, Reward=1
Step 2: Action=1, Reward=3
Step 3: Action=2, Reward=5
Step 4: Action=3, Reward=7
Step 5: Action=4, Reward=9
Step 6: Action=4, Reward=9
Step 7: Action=4, Reward=9
Step 8: Action=4, Reward=9
Step 9: Action=4, Reward=9
Step 10: Action=4, Reward=9
Step 11: Action=4, Reward=9
Step 12: Action=4, Reward=9
Step 13: Action=4, Reward=9
Step 14: Action=4, Reward=9
Step 15: Action=4, Reward=9
Step 16: Action=4, Reward=9
Step 17: Action=4, Reward=9
Step 18: Action=4, Reward=9
Step 19: Action=4, Reward=9
Step 20: Action=4, Reward=9
Total Reward (UCB): 160
>>>
```

Right Window Code:

```
total_reward += reward
print(f"Step {t+1}: Action={action}, Reward={reward}")
print("Total Reward ( $\epsilon$ -Greedy):", total_reward)
print("\n===== UCB =====")
counts = [0] * num_arms
values = [0.0] * num_arms
total_reward = 0
for arm in range(num_arms):
    reward = true_rewards[arm]
    counts[arm] = 1
    values[arm] = reward
    total_reward += reward
    print(f"Step {arm+1}: Action={arm}, Reward={reward}")
for t in range(num_arms, steps):
    ucb_values = []
    for arm in range(num_arms):
        bonus = math.sqrt((2 * math.log(t + 1)) / counts[arm])
        ucb_values.append(values[arm] + bonus)
    action = ucb_values.index(max(ucb_values))
    reward = true_rewards[action]
    counts[action] += 1
    values[action] += (reward - values[action]) / counts[action]
    total_reward += reward
    print(f"Step {t+1}: Action={action}, Reward={reward}")
print("Total Reward (UCB):", total_reward)
```

Result:

The Multi-Armed Bandit problem was successfully implemented using ϵ -Greedy and UCB algorithms. The cumulative rewards obtained by both methods were evaluated successfully.

Experiment 8: Reward Visualization in Reinforcement Learning

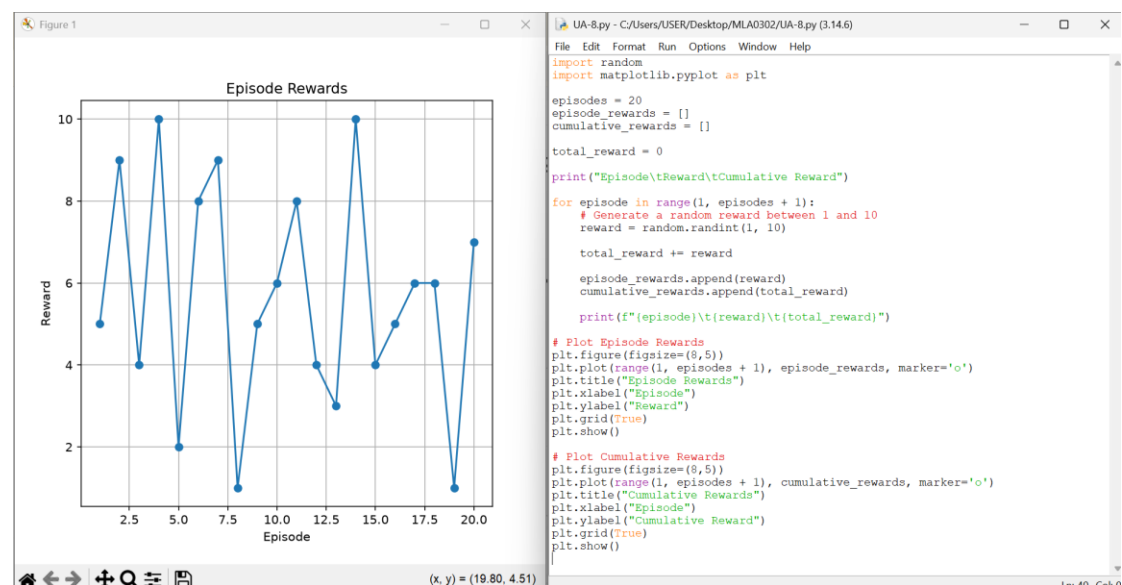
Aim:

To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Procedure :

1. Simulate an RL agent for multiple episodes.
2. Generate rewards for each episode.
3. Calculate cumulative rewards.
4. Store the reward values.
5. Plot episode rewards using Matplotlib.
6. Plot cumulative rewards using Matplotlib.
7. Analyze the generated graphs.

Output :



Result:

The RL agent's rewards were successfully visualized using Matplotlib. The graphs clearly illustrated episode rewards, cumulative rewards, and learning performance.

Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Procedure :

1. Import TensorFlow, Keras, and NumPy.
2. Prepare sample state-value training data.
3. Build a feed-forward neural network.
4. Compile the model using the Adam optimizer.
5. Train the neural network.
6. Predict the value of a new state.
7. Display the predicted value.

Output :

```
from tensorflow import keras
from tensorflow.keras import layers

# Sample state data (4 features)
x = np.array([
    [0.1, 0.2, 0.3, 0.4],
    [0.5, 0.6, 0.7, 0.8],
    [0.2, 0.3, 0.4, 0.5],
    [0.6, 0.7, 0.8, 0.9],
    [0.3, 0.4, 0.5, 0.6],
    [0.7, 0.8, 0.9, 1.0]
], dtype=np.float32)

# Target values (Value Function)
y = np.array([1, 2, 1.5, 3, 2, 3.5], dtype=np.float32)

# Build Neural Network
model = keras.Sequential([
    layers.Dense(16, activation='relu', input_shape=(4,)),
    layers.Dense(8, activation='relu'),
    layers.Dense(1)
])

# Compile Model
model.compile(optimizer='adam',
              loss='mse',
              metrics=['mae'])

# Train Model
model.fit(x, y, epochs=5, verbose=1)

# Predict value for a new state
new_state = np.array([0.4, 0.5, 0.6, 0.7], dtype=np.float32)
predicted_value = model.predict(new_state)

print("\nPredicted State Value:")
print(predicted_value)
```

```
... Epoch 1/5
1/1 ----- 1s 1s/step - loss: 3.6356 - mae: 1.7781
Epoch 2/5
1/1 ----- 0s 144ms/step - loss: 3.5562 - mae: 1.7577
Epoch 3/5
1/1 ----- 0s 75ms/step - loss: 3.4775 - mae: 1.7372
Epoch 4/5
1/1 ----- 0s 78ms/step - loss: 3.3994 - mae: 1.7167
Epoch 5/5
1/1 ----- 0s 108ms/step - loss: 3.3219 - mae: 1.6961
1/1 ----- 0s 118ms/step

Predicted State Value:
[[0.4912835]]
```

Result:

A feed-forward neural network was successfully developed using TensorFlow and Keras. The model learned the sample data and predicted the value function for a new state.

Experiment 10: Mini Reinforcement Learning Project Using CartPole**Aim:**

To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

Procedure :

1. Import Gymnasium, TensorFlow, Keras, and NumPy.
2. Create the CartPole-v1 environment.
3. Build a neural network model.
4. Initialize the RL agent.
5. Execute multiple training episodes.
6. Record the episode rewards.
7. Calculate the success rate and evaluate the learning performance.

Output :

```

    success = 0

    print("CartPole Reinforcement Learning")

    for episode in range(episodes):

        state, info = env.reset()
        state = np.reshape(state, [1, state_size])

        total_reward = 0

        while True:

            # Random action (for demonstration)
            action = env.action_space.sample()

            next_state, reward, terminated, truncated, info = env.step(action)

            total_reward += reward

            next_state = np.reshape(next_state, [1, state_size])

            target = model.predict(state, verbose=0)
            target[0][action] = reward

            model.fit(state, target, epochs=1, verbose=0)

            state = next_state

            if terminated or truncated:
                break

        print(f"Episode {episode+1}: Reward = {total_reward}")

        if total_reward >= 195:
            success += 1

    env.close()

    print("\nSuccess Rate:", (success / episodes) * 100, "%")

*** CartPole Reinforcement Learning
Episode 1: Reward = 32.0
Episode 2: Reward = 14.0
Episode 3: Reward = 13.0
Episode 4: Reward = 38.0
Episode 5: Reward = 20.0
Episode 6: Reward = 23.0
Episode 7: Reward = 12.0
Episode 8: Reward = 16.0
Episode 9: Reward = 21.0
Episode 10: Reward = 25.0

Success Rate: 0.0 %

```

Result:

A basic Reinforcement Learning agent was successfully developed for the CartPole environment. The agent interacted with the environment, collected episode rewards, and its performance was evaluated using the success rate and reward obtained.

